

GPU-centric Communication on NVIDIA GPU Clusters with InfiniBand: A Case Study with OpenSHMEM

S. Potluri, A. Goswami, D. Rossetti and C. J. Newburn
NVIDIA Corporation
Santa Clara, California 95050
Email: spotluri,agoswami,drossetti,cnewburn@nvidia.com

M. Gorentla Venkata and N. Imam
Oak Ridge National Laboratory
Oak Ridge, Tennessee 37830
Email: manjugv,imamn@ornl.gov

Abstract—GPUs have become an essential component for building compute clusters with high compute density and high performance per watt. As such clusters scale to have 1000s of GPUs, efficiently moving data between the GPUs becomes imperative to get maximum performance. NVSHMEM is an implementation of the OpenSHMEM standard for NVIDIA GPU clusters which allows communication to be issued from inside GPU kernels. In earlier work, we have shown how NVSHMEM can be used to achieve better application performance on GPUs connected through PCIe or NVLink. As part of this effort, we implement IB verbs for Mellanox InfiniBand adapters in CUDA. We evaluate different design alternatives, taking into consideration the relaxed memory model, automatic memory access coalescing and thread hierarchy on the GPU. We also consider correctness issues that arise in these designs. We take advantage of these designs transparently or through API extensions in NVSHMEM. With micro-benchmarks, we show that a Nvidia Pascal P100 GPU is able saturate the network bandwidth using only one or two of its 56 available streaming multiprocessors (SM). On a single GPU using a single IB EDR adapter, we achieve a throughput of around 90 million messages per second. In addition, we implement a 2dstencil application kernel using NVSHMEM and compare its performance with a CUDA-aware MPI-based implementation that uses GPUDirect RDMA. Speedups in the range of 23% to 42% are seen for input sizes large enough to fill the occupancy of Nvidia Pascal P100 GPUs on 2 to 4 nodes indicating that there are gains to be had by eliminating the CPU from the communication path when all computation runs on the GPU.

I. INTRODUCTION

NVIDIA GPUs have gained wide adoption in supercomputing clusters for their high compute density and power efficiency. Compute Unified Device Architecture (CUDA) [1] is the defacto programming framework for NVIDIA GPUs. Application developers typically use CUDA in conjunction with high-level programming models like the Message Passing Interface (MPI) [2] to develop codes that run across multiple nodes. Several MPI implementations [3], [4] provide “CUDA-Awareness” by which standard MPI interface can be used to move data between GPUs, within a node or across nodes. This also allows MPI runtimes to optimize GPU-GPU communication transparently for the user. In MPI+CUDA applications, enforce computation and communication phases to be sepa-

rated with CPU-GPU synchronization at their boundaries to resolve dependencies.

There is an overhead associated with launching compute kernels on the GPU. Alternating compute and communication phases result in recurring kernel launch overheads. The synchronizing activity on the GPU before MPI communication leads to under-utilization of GPU resources during communication and synchronization phases. Similarly, there is under-utilization of the network when computation is in progress. Some of these issues are handled by restructuring the application code to overlap independent compute and communication phases, using CUDA streams. These optimizations make the application code complex and their benefits usually diminishes as the problem size per GPU becomes smaller [5].

One of the approaches to address these limitations is by adding the capability to initiate communication on the GPU. This can result in long running kernels, providing better utilization of the GPU while avoiding launch and synchronization overheads. Communication from within CUDA kernels enables a highly concurrent, finer-grained communication model in applications. In earlier work, we had presented NVSHMEM, an OpenSHMEM library for device-initiated communication but limited to GPUs interconnected by PCIe or NVLink. In this paper, we explore and evaluate device-side OpenSHMEM like communication over InfiniBand. We prototype user space and kernel space libraries to support InfiniBand Verbs API from inside CUDA kernels.

Researchers have implemented prototypes of IB verbs for GPUs in prior work. Oden et al. had observed that IB verbs are not suited for GPU-initiated communication [6]. Daoud et al. have shown ways to work around overheads and achieve better performance with GPU verbs [7]. We push the start-of-the-art in this paper. We consider GPU-specific features like implicit memory coalescing as well as the overheads of the relaxed memory model in the context of GPU verbs. We show how the location of IB command and completion queues can lower the strength of required memory barriers to improve performance. We revisit optimizations like batching and templating from earlier work in relation to the memory model. We present a discussion on how these optimizations can be used in an Open-

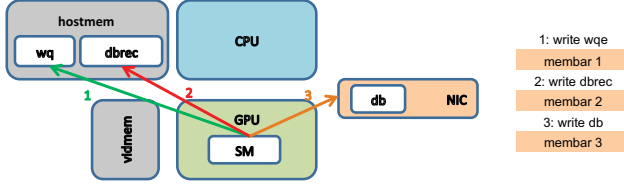


Fig. 1. Steps of posting a command to the network adapter through IB verbs

SHMEM runtime either transparently or through appropriate API extensions. We present a detailed micro benchmark level evaluation of our designs and discuss porting of a 2dstencil mini-app to use GPU verbs through NVSHMEM. We make the following key contributions through this paper:

- We design GPU-side IB verbs considering GPU memory model, memory access coalescing and concurrency
- We design OpenSHMEM using the proposed GPU-side IB verbs
- We present a detailed micro-benchmark level evaluation of our designs and present the porting of a 2dstencil mini-app from CUDA+MPI to NVSHMEM over GPU Verbs.

II. BACKGROUND

A. InfiniBand Verbs

InfiniBand (IB) has become one of the most commonly used interconnects in High Performance Clusters. IB verbs are the user-mode API using which an InfiniBand adapter can be programmed. In this paper, we are primarily interested in the API using which one can initiate a data movement operation in IB and poll for completion of that operation. We have implemented a prototype of these two APIs in CUDA so that kernels running on an NVIDIA GPU can directly issue data transfer operations to the IB adapter and can poll for their completion. The user-level protocol for IB is defined around queues. Initiating communication operation on InfiniBand involves preparing a Work Queue Element (WQE) in a memory that is accessible by the NIC DMA engine. The CPU then sends a message to the DMA engine indicating this purpose. There is an intermediate step where a pointer to the command is written to a buffer called the doorbell record. This is used by the adapter to access the command if the "ringing of the doorbell" is missed due to congestion or conflict. Figure 1 shows these steps pictorially. This means that appropriate memory barriers have to be placed between the three steps to ensure ordering, on systems with relaxed memory models. The completion of a communication operation is detected by polling on a Completion Queue (CQ). On completing an operation, the network adapter generates a Completion Queue Entry (CQE) to provide status of a WQE. This paper provides an implementation of IB verbs for CUDA kernels.

B. OpenSHMEM

OpenSHMEM is a PGAS library interface specification. It presents a PGAS view of execution contexts and memory

model. The execution contexts in OpenSHMEM is an OS process identified by an integer called Processing Element (PE). An OpenSHMEM program has private address space per PE and shared address space across PEs. The shared address space in OpenSHMEM is presented as symmetric objects, which are accessible by all PEs in an OpenSHMEM program. The symmetric objects are allocated using a collective allocation operation in OpenSHMEM.

OpenSHMEM provides routines for communication and synchronization with other PEs. The communication in OpenSHMEM is primarily one-sided. It provides many variants of Put, Get and Atomic operations to access and modify symmetric data objects that are located on remote PEs. It also provides ordering and collective communication APIs.

III. GPU-INITIATED COMMUNICATION USING OPENSHEMEM

NVSHMEM presents a programming approach using GPU-initiated communication which is motivated by the throughput oriented architecture of the GPU. The GPU is designed to support tens of thousands of threads to achieve maximum parallel throughput. These threads are extremely light weight and thousands of threads are queued up for work (in groups called warps). While one warp waits on a memory access, another warp starts executing in its place. As a separate set of registers is allocated for all active threads, there is no need for swapping of registers or state. As a consequence, this execution model has inherent latency hiding capabilities with minimal scheduling overheads. With the increasing amount of parallelism, GPU architectures can have enough state to hide latencies not only to local GPU device memory but also to remote GPU memory over a network. GPU-initiated communication can take advantage of this inherent capability of the GPU hardware. Further, this improves programmability as developers will not have to rely on a hybrid model to orchestrate and overlap between different phases of the application. The communication initiated and synced from within CUDA kernels will not only reduce the reliance on the CPU, additionally it also avoids existing synchronization overheads that limit the strong scaling. Using PGAS programming model such as OpenSHMEM suits GPU-initiated communication better. The one-sided communication model of OpenSHMEM requires only the caller (origin) of the interface to be active, which matches the massive parallelism and dynamic scheduling model on the GPU.

IV. DESIGN AND IMPLEMENTATION

A. GPU-side IB verbs:

Other researchers have prototyped IB verbs for the GPU earlier [6], [7]. In this work, we go further in discussing how GPU features like warp-level coalescing and the relaxed GPU memory model can impact their performance. We also discuss how GPU-side verbs can be used to implement the OpenSHMEM communication model. We explain how performance optimizations like templating and batching of IB verbs requests can be used transparently when implementing

an OpenSHMEM like model. Further, we consider receiver-side consistency issue that was left out by earlier work and address it in the context of SHMEM. Finally, we present the implementation of 2DStencil, a mini-app which is representative of typical iterative HPC applications, using GPU-side IB verbs. A background on IB verbs and the sequence of operations that constitute the issuing of an IB command to the network adapter is explained in detail under Section II-A.

Warp-level Coalescing: The grouping of threads into warps in CUDA is not only specific to computation but is also relevant when accessing memory. When threads within a warp write to consecutive addresses in memory, these accesses are automatically coalesced into larger transactions (typically equal to the cache-line size). This happens whether the accesses are to the GPU global memory, CPU/Host memory or to a memory of a peer PCIe device. Issuing operations in IB involves preparing a command and writing it to the command queue (step 1 shown in Figure 1). An optimization would be to inline data with the command and write it to the command queue as part of the above operation. There is a network adapter-specific limit on the size of data that can be inlined. Another possible optimization is when ringing the doorbell (step 3 shown in Figure 1). This involves writing the complete command to the doorbell register (instead of just writing the first 8 bytes containing the location of the command). This saves the adapter from having to read the command from memory. The operation of writing the command into the queue will involve a write to Host or GPU memory depending on where the command queues had been created. The write to the doorbell involves a write over PCIe to a register on the network adapter. In both these cases, threads within the same warp can be used to collectively issue the writes and thus generate a single transaction to memory or over PCIe.

The current OpenSHMEM standard provides only APIs that can be called by a single thread of execution. To take advantage of parallelism on the GPU and to benefit from the coalescing capabilities, we have extended the SHMEM API by adding a thread group parameter to them. For example, the modified `shmemx_put` API will look as follows:

```
void shmemx_put(TYPE *dest, const
    TYPE *source, size_t nelems,
    int pe, int thread_group);
```

Thread groups can be threading model specific. In the context of CUDA, they can be `THREAD`, `WARP`, `CTA` or `GRID`. We expect device-side communication API to be called at the `WARP` granularity or higher. This will allow us to take advantage of coalescing in SHMEM, as explained above. In our examples, we have used `WARP`-level SHMEM operations.

Impact of GPU Memory Model: NVIDIA GPUs have a highly relaxed memory model which means that reads or writes issued by a CUDA thread can be reordered except when they are to the same address or when there is an appropriate data/control dependency or when there is an appropriate memory barrier. Memory barrier in PTX (Parallel Thread Execution ISA for NVIDIA GPUs) has a scope (`sys`, `gl`, `cta`) qualifier

wqe	dbrec	membar 1	membar 2	membar 3
sysmem	sysmem	membar.sys	membar.sys	-
vidmem	sysmem	membar.gl	membar.sys	membar.sys
vidmem	vidmem	membar.gl	membar.gl	-

Fig. 2. Memory barriers required based on the location of command queues and doorbell record

which specifies the entities to which the operations separated by the barrier will become visible in order. For example `membar.sys` guarantees ordering is visible to all other threads in the GPU and all clients including those interacting with the GPU over PCIe (IB adapter in our scope). Posting an IB operation involves three writes (steps 1,2 and 3 in Figure 1) which are required to appear in order to the network adapter for correct execution. As explained in Section II-A, all these writes happen to different memory locations: command queue, doorbell record and doorbell register. A CUDA thread has to insert memory barriers to guarantee their ordering. Table in Figure 2 shows the memory barriers required based on which memories the command queues and doorbell record are located in. When both command queue and doorbell record are in host memory, `membar.sys` is required between updates to these locations and before the update to doorbell, to guarantee ordering. Considering the next combination, when command queue is in global memory, a `membar.gl` is sufficient in place of `membar1`. This is based on the property that `membar.gl` guarantees visibility ordering for updates to global memory w.r.t the PCIe endpoint on the GPU. Any accesses coming from the NIC to global memory are serviced by this PCIe endpoint and hence observe the same ordering. However, this is not guaranteed by the ISA specification and can change on future architectures. A `membar.sys` is required after update to doorbell record, when it is in host memory. In this case, a `membar.sys` is required after "ringing the doorbell" in order to prevent it from getting reordered with the write to doorbell record from the next transaction. This can result in the next command being picked up before it has been fully written. When both command queue and doorbell record are in GPU memory, `membar.gl` is sufficient in place of both `membar1` and `membar2`.

The ideal location of command queue and doorbell record will depend on where the communication is initiated from. We expect that OpenSHMEM runtimes can maintain separate endpoint resources for use from the GPU. There have been recent proposals for having multiple communication contexts in OpenSHMEM, each context having different usage characteristics (e.g. single threaded vs multi-threaded). Whether a given context will be used from the host or the device can be a new parameter that allows the runtime to allocate IB resources appropriately.

Completion Polling: While Daoud et al., in their earlier work, have experimented with placing data structures on Host or Device memories, they have not compared the use of a CPU thread to poll for completions and have the GPU threads poll for completions inside the CUDA kernels. We have im-

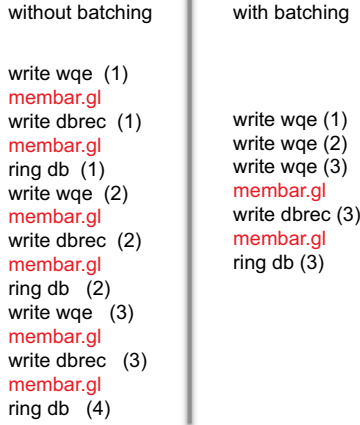


Fig. 3. Sequence of operations with and without batching

plemented these alternatives and compared their performance in the later sections. These design choices are transparent to the user in terms of the programming model. Even when CPU is used for polling IB completions, threads on the GPU can query for completion of SHMEM communication API from inside a CUDA kernel (for e.g., by calling `shmem_quiet`). The CPU in this case will act as a proxy that polls completions and signals the GPU thread when done.

Batched Posts: Though the strength of memory barriers can be reduced when the IB data structures are in device memory, they still form a reasonable portion of time taken for initiating IB communication operations on the GPU. This is explained with performance numbers in the following section. In order to reduce the impact of memory barriers on throughput, we have implemented a design for batching multiple operations before posting them to the IB adapter. Figure 3 shows the difference between issuing a sequence of operations without batching and issuing them with batching. It reduces the number of memory barriers that are required. Assuming the operations are happening on the same reliable endpoint, updating the doorbell record and ringing the doorbell with values for the last operation will also trigger the previous operations that were not issued. This reduces the number of writes to doorbell record and doorbell register as well. The benefit from reduced writes had been presented earlier in work by Daoud et al. Specifically, QP on GPU memory + no inline + thread in Tables I and II of Section V closely represent the work done by Daoud et al. In this work, we have considered other possible scenarios as well as the impact on memory barriers for a given scenario.

It is typical for OpenSHMEM applications to issue a number of operations before blocking for their completion (using `shmem_quiet`). This allows for the above optimization to be applied transparently in the OpenSHMEM runtime. All pending operations have to be issued and polled for when an `nvshmem_quiet` is called.

Templated Operations: Owing to the low serial performance of GPU cores, the cost of issuing the operation forms a

considerable portion of the total communication time for small messages. This can be reduced if the command queues can be prepared ahead of time. However, this requires knowledge of local and remote memory addresses, data transfer sizes, etc. Such an optimization is possible when implementing synchronizing operations like barriers which can use internal memory and repeatedly perform the same operations. We apply this in our implementation of the 2Dstencil mini-app as explained later.

While this cannot be applied to OpenSHMEM communication API in the current standard, an extension to allow persistent communication handles can benefit from this. Persistent handle can find use in fixed-grid iterative applications where each process sends data from the same memory locations to the same set of processes in each iteration.

Target-side Consistency: Threads in a GPU kernel, that is already running, are not guaranteed to see the updates to memory done by the CPU or other devices in a specific order and within a specific time interval. CUDA does not provide a way to enforce this consistency which we refer to as target-side consistency. This problem was highlighted in earlier work by Daoud et al. but the authors do not offer a solution. We handle this consistency problem by making sure that completion of communication and the use of the received data happen across a kernel boundary. For example, in our implementation of 2Dstencil, we have a `shmem_barrier` executed at the end of each iteration which guarantees the completion of all the communication operations issued before it returns. This marks the end of a CUDA kernel as well. The data received is accessed in the kernel launch from the next iteration. This launch provides the required consistency guarantees. Launching one kernel per iteration does not necessarily mean that there has to be a synchronization with the CPU at the end of each iteration. A sequence of iterations (kernel launches) can be queued up on a CUDA stream which execute asynchronously w.r.t the CPU. A CUDA kernel launch has software and hardware overheads associated with it. Queuing multiple kernels on a stream overlaps these stages, thereby providing latency hiding. Applications that require a convergence check will have to launch one or more iterations speculatively in order to benefit from the queuing. These benefits are explained in more detail in the following section.

B. 2D stencil miniapp

We have implemented two versions of a 2D stencil miniapp, one using MPI and the other using SHMEM on top of GPU Verbs. In the MPI version, all communication is initiated from the CPU through the MPI library. On the other hand, in the SHMEM version, communication is initiated directly from the GPU using GPU Verbs. This avoids kernel launch overheads that is critical for small input sizes when the kernel execution time is low. Moreover, synchronization between the MPI library on the CPU and data produced on the GPU memory by the CUDA kernels are necessary to ensure data consistency is avoided.

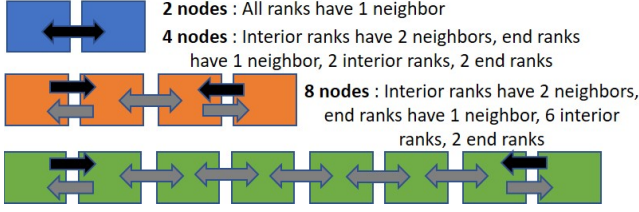


Fig. 4. 1D process grid layout

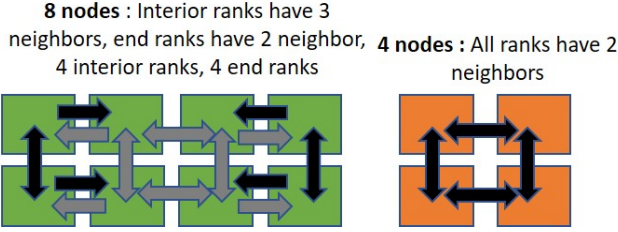


Fig. 5. 2D process grid layout

1) *Data partitioning*: Each rank or PE gets a square tile of the data, distributed equally among all the ranks or PEs. The processes are arranged in all possible 1D and 2D layouts. Figure 4 shows the possible 1D grid layouts (transpose grid layouts are not shown). In this case, two is the maximum number of neighbors that any rank ever communicates with when there are 3 or more processes.

2D grid layouts are only possible for 2 or more nodes as shown in Figure 5 (transpose layout for 8 nodes is not shown). In this case, the processes at the corners communicate with *two* other neighbors, the ones at the borders communicate with *three* other neighbors and the ones in the interior communicate with *four* other neighbors. Thus, the communication per process does not scale.

2) *Granularity of data transfer*: Each CTA is assigned a section of the boundary regions to work on. Once the boundary computation is done, it needs to send the data to the remote PE. The granularity at which the boundaries are exchanged can vary depending on the application. Proposals are in place to allow users to specify synchronization granularity to the SHMEM remote memory access APIs. For example, a CTA can choose to perform a put each time it slides to a new position on the boundary. This requires intra-CTA synchronization if multiple warps engage in data transfer and can be quite easily accomplished by the `__syncthreads()` primitive in CUDA. However, the overhead of posting many small-sized messages from the GPU can hurt performance. Alternatively, the CTA can use batching to avoid the overhead of multiple small writes and send the entire boundary region assigned to it as a single message. Finally, a CTA can wait until all other CTAs are done computing their respective segments of the boundary and then combine all messages directed to a given neighbor into a single message issued by one of the CTAs. If there are multiple neighbors to send data to, some number of CTAs can be chosen and assigned the task of transferring to

specific neighbors.

V. EVALUATION

The evaluation can be divided into two parts.

- 1) Micro-benchmark experiments to study latency, throughput and bandwidth achieved by GPU verbs.
- 2) Experiments with a 2D stencil mini-app to show application-level performance impact of using the GPU verbs library.

A. Micro-benchmarks

All experiments in this section were conducted on two 14-core Intel Xeon Broadwell (E5-2680v4) machines with NVIDIA Tesla P100 GPUs connected by Mellanox ConnectX-4 MT27700 EDR InfiniBand (IB) adapters. The NIC and the GPU were connected by a PLX 9797 PCIe switch bypassing the PCIe Host Bridge. As a result, the entire 12 GB/s bandwidth is available for data transfer between the GPU and the NIC. Ubuntu 14.04 and CUDA driver version 375.66 were installed on the machines. The OFED version 3.4 was used.

1) *Latency*: In the first latency experiment, we measure the latency to post an RDMA write. *This corresponds to the latency of a PE calling `shmem_put_nbi`*. The configurations used are QP on GPU or system memory, inline v/s non-inline configuration of the NIC and using a warp v/s using a thread to perform the post. Table I shows the latency to post an RDMA write from the GPU. The minimum latency of 2.26 us is obtained when (1) QP is on the GPU memory and (2) a warp is used to execute the post.

In general, the warp versions are faster than the thread version because updating the WQE involves a 64 byte write to the GPU or the system memory. In the thread version, this gets serialized. But the warp version is able to do this concurrently using more than one thread thereby reducing the latency. Also, when the QP is on the GPU memory, the latency is lower as expected because accessing to system memory over PCIe is slower. Inlining doorbell does not have effect on the performance when WARP is used as the number of packets issued over PCIe does not change due to coalescing (one packet is collectively written by the WARP). Inlining hurts post latency in the thread case (3.26us compared to 2.56us), as the single thread has to issue more writes to posed the inlined command which is longer.

In the second latency experiment, the ping-pong latency is measured. *This corresponds to one PE calling `shmem_put` followed by `shmem_wait` and the other PE calling `shmem_wait` followed by `shmem_put`*. This is calculated by measuring the total time to (1) post a write of 4 bytes from one GPU and (2) poll on memory to observe the write on the second GPU (3) post a write back to the first GPU and (4) poll and observe the write on the first GPU. This memory pingpong latency is divided by 2. The results are shown in Table II. In this case too, the lowest latency of 3.16 us is observed when the (1) QP is on the GPU memory, (2) both the data and the doorbell are inlined, and (3) a warp is used to post the write. Here inlining data and doorbell write helps because the latency for NIC to

TABLE I
POST LATENCY FOR GPU INITIATED RDMA WRITE

Latency (us)							
QP on System Memory				QP on GPU Memory			
No inline		Inline data and DB		No inline		Inline data and DB	
Thread	Warp	Thread	Warp	Thread	Warp	Thread	Warp
7.14	6.04	7.14	6.08	2.56	2.26	3.26	2.29

fetch data and command from memory is considered in this test. Inlining avoids this fetch.

In the third latency experiment, we measure the ping latency by recording the total time to post a RDMA write and poll for a completion, from a thread on the GPU or on the CPU. When the polling is done on the GPU, the CQ can be placed on the GPU memory or the system memory. But for CPU-side polling, we only consider QP and CQ to be both on the system memory. When polling is done on the CPU, the latency includes the time for the CPU to signal the GPU thread after the completion has arrived. *This test corresponds to the latency of a PE calling `shmem_put` followed by `shmem_quiet`.* The results are shown in Figure 6. The minimum latency is observed when (1) both the QP and the CQ are placed on the GPU, (2) a GPU thread polls for completions, (3) both the data and the doorbell are inlined, and (4) a warp is used to post the write. For a 8 byte message, it is 4.97 us.

The breakdown of the ping latency across post and the poll (for the best configuration above) with increasingly relaxed runtime safety checks is shown in Table III. By default, all guards such as flow control and error handling are enabled. The software flow control ensures that there is no overrun of queues. Otherwise, it is possible that the WQ entries are overwritten before the corresponding completion arrives. Eliminating the flow control makes the posting faster. Finally, the error checks are meant to catch and report readable errors when a NIC operation fails. Removing error checks helps to speedup polling.

The preparation of a WQ entry is something that can be reused or done beforehand. But updating the doorbell record and ringing the doorbell register on the NIC are two tasks that need to be done at the time of every post. Moreover, by separating the two, a memory barrier that is otherwise needed can be avoided. Table IV gives the breakdown of the ping latency when prepare and post are separated v/s combined into a single API.

2) *Throughput and bandwidth*: In the next set of experiments, we evaluated two different aspects in addition to the effect of where QP/CQ is placed and who polls for completions.

- What is the effect of using more warps or threads to transfer data?
- How much does amortizing latency over successive posting of writes help to improve the achieved throughput?

Each of these are described below.

a) *Number of warps or threads used for posting writes*:

The bandwidth and throughput achieved by different occupancy levels are shown in Figure 7 and Figure 8 when (1) a

single warps on the GPU is used to post the writes, (2) when one warp on each SM is used, (3) when four warps on each SM is used to exercise all QPs available on the NIC, and (4) when all sixteen warps available on the GPU are used. In addition, the placement of the QP is varied between the GPU memory and system memory. Also, the polling for completions is done either by the GPU or the CPU. Finally, successive writes are batched into a single post.

Throughput discussion: Using a single warp or a single thread, the peak throughput is much lower than what is achieved by a single thread on a CPU. This is expected because single GPU thread or warp execution is known to be slower than a CPU thread. In case of the CPU, multiple queue pairs (QP) are used by the CPU thread to improve the throughput thereby utilizing the parallelism of the NIC. In case of the GPU, rather than have a single warp or thread use multiple QPs, we look to use more warps to increase the throughput achieved. The highest message rate achieved is around 16 million messages per second when small writes (≤ 128 bytes) are posted by sixteen warps on each SM on the Tesla P100 GPU consisting of 56 SMs with the QP placed on the GPU memory. *Bandwidth discussion*: With a single warp on the GPU, only messages of size 128KB or higher can saturate the available bandwidth of 12 GB/s. In contrast, a CPU thread reaches saturation with only 4KB sized messages. Using all the warps on one SM, the GPU is able to saturate the available bandwidth with 4KB messages similar to a CPU thread. Dedicating one SM for communication on a Tesla P100 GPU with 56 SMs amounts to a small fraction of the available compute resources. This leaves plenty of resources to run other warps for doing compute.

b) *Batched posting of writes*: We implement the batching design explained in Section IV-A where multiple ops are buffered in the command queue and triggered at once. *Throughput discussion*: Batching helps improve the throughput significantly, showing a peak message rate of around 90 Million messages per sec. The drop in message rate after 8 bytes is because of a limit in the size of data that can be inlined. This is a limit in our verbs implementation and can be fixed. Most IB hardware supports an inline size of 128 bytes or more.

B. 2D-stencil miniapp

All experiments in this section were conducted on a eight node cluster of 16-core Intel Xeon Haswell (E5-2698v3) machines with NVIDIA Tesla P100 GPUs connected by Mellanox Connect-IB MT27600 FDR InfiniBand (IB) adapters. CentOS 7.3 and CUDA driver version 375.39 were installed on the

TABLE II
PING-PONG LATENCY FOR GPU INITIATED RDMA WRITE

Latency (us)							
QP on System Memory				QP on GPU Memory			
No inline	Inline data and DB	No inline	Inline data and DB	No inline	Inline data and DB	No inline	Inline data and DB
Thread	Warp	Thread	Warp	Thread	Warp	Thread	Warp
7.28	6.26	7.24	5.63	4.37	4.14	4.25	3.16

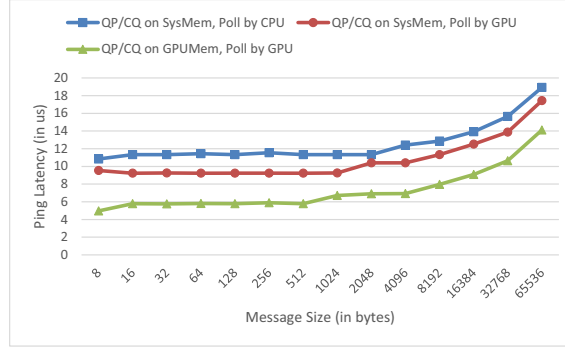


Fig. 6. Ping latency using different placements of QP/CQ and polling agents

TABLE III
PING LATENCY BREAKDOWN WHEN RUNTIME SAFETY CHECKS ARE RELAXED

Message (bytes)	Latency (us)					
	All checks present		No flow control		No error handling	
	Post	Poll	Post	Poll	Post	Poll
16	3.18	3.46	2.95	3.45	2.87	2.71
1024	3.18	3.57	2.73	3.31	2.88	2.85
65536	2.96	11.44	2.69	11.31	2.67	10.72

TABLE IV
PING LATENCY BREAKDOWN ACROSS PREPARE, POST AND POLL

Latency (us)		
Prepare	Post	Combined prepare and post
1.02	1.51	2.96

machines. The OFED version used was 4.0. MVAPICH2-GDR 2.2, a CUDA-aware MPI implementation with GPUDirect RDMA support, was used for comparison and also as the baseline.

The number of neighboring ranks with whom any given rank exchanges data depends on both the total number of ranks as well as how the ranks are placed in the 2D grid and whether the rank is on the edge/corner or interior of the grid. For 2 nodes, each rank will always have 1 peer. But for 4 and 8 nodes, it can vary as described in Section IV-B. The 2D-stencil miniapp represents an important category of HPC application that do near-neighbor communication. Data is transferred to a maximum of four neighboring ranks (*top*,

right, *bottom*, *left*) where all the ranks are arranged in a 2D grid. Each rank is placed on a different node and uses a single GPU. Each GPU works on the same amount of data which is a square matrix of single precision floating point numbers. The number of neighboring ranks with which any given rank exchanges data depends on both the total number of ranks as well as how the ranks are placed in the 2D grid and whether the rank is on the edge/corner or interior of the grid. For 2 nodes, each rank will always have 1 peer. But for 4 and 8 nodes, it can vary as described in Section IV-B. For higher number of nodes, the communication due to the interior nodes start to dominate. Moreover, due to the near-neighbor communication pattern, the bytes of communication per rank does not grow.

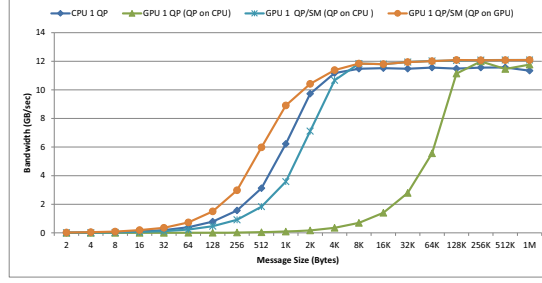


Fig. 7. Bandwidth using different number of QPs from the GPU

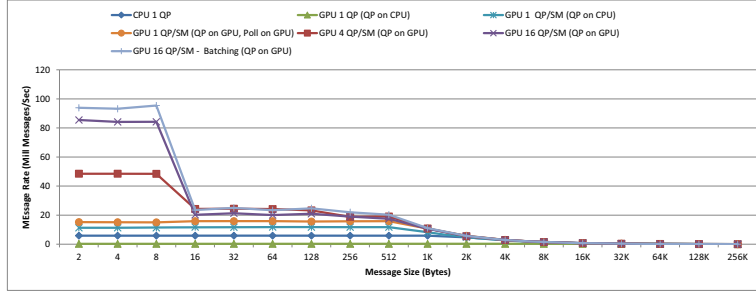


Fig. 8. Throughput (messages per second) for different configurations

Strong scaling is emulated by varying the size of the matrix per GPU. As such, we argue that the emulation of strong scaling reasonably demonstrates the effect of reduced compute per GPU on overall performance.

The baseline version was implemented using MPI and requires launching five different kernels to interleave the MPI communication happening on the CPU and the computation/data (un)packing that runs on the GPU. In order to support synchronization and memory ordering semantics of the SHMEM programming model without causing a deadlock, the total number of CTAs that any kernel using GPU-initiated communication can be launched with must not exceed the occupancy of the kernel. For the MPI version of the 2D-stencil

miniapp, the size of the CTA was chosen to be 256 threads which gives optimal performance on P100 GPUs. As a result, the kernels in the MPI version has occupancy of 448 CTAs. In other words, a maximum of 448 CTAs of the kernels in the MPI version can be running concurrently. The number of CTAs that gets launched is controlled by the matrix size per GPU. The occupancy is not filled until the matrix size per GPU exceeds 512x512. For sizes 512x512 and below, each active CTA executes just a single iteration of the kernel. So time per iteration to execute the kernels is comparable to the time to launch the kernels. This gives a relevant window of problem sizes that is important for strong scaling.

We implement a version of 2D stencil which uses GPU

Verbs to post RDMA writes from inside the kernel avoiding the overheads of multiple kernel launches as well as eliminating the kernels that handle data (un)packing necessary for MPI communication. Moreover, the polling for completions as well as enforcing a barrier between ranks to ensure data consistency is done from the GPU reducing the overhead further compared to CPU-side polling and enforcing the barrier using MPI on the CPU. The fused compute (interior and boundary) and communication (boundary exchange) kernel, however, consumes more resources on the P100 GPU than the five specialized kernels in the MPI version. This lowers the occupancy to 224 CTAs with the same CTA size (256 threads). Due to the lower occupancy of the GPU Verbs version of the kernel, a matrix size exceeding 256x256 per GPU activates all CTAs. So the strong scaling window for the MPI and the GPU Verbs versions are not identical. We will see next that this causes performance artifacts around the occupancy size for the two version.

The occupancy of the two versions of the kernels is important also because it determines if overlap is possible between different phases in an iteration. Before occupancy is reached, some or all of the interior computation can be overlapped with the boundary compute and exchange by specializing warps to take care of these tasks in the GPU Verbs version. The MPI version achieves the same by launching the interior compute on a separate CUDA stream from one that it uses for boundary compute and data (un)packing.

The speedup of the GPU Verbs compared to the MPI version is shown in Figure 9. When the input matrix size is lower than the occupancy, the GPU Verbs version is always better than the MPI version. The speedup ranges from 24% to 42%. The performance advantage of the GPU verbs version diminishes as the number of CTAs per GPU approaches occupancy. For the window of matrix size of 1Kx1K to 2Kx2K, the performance of the GPU Verbs version dips below the MPI version by around 10%. This is due to differences in occupancy of the MPI and the GPU Verbs versions of the kernels. If we launch the same number of CTAs in both the versions, this performance dip goes away. However, that would imply artificially limiting the performance of the MPI version which we consciously avoided. Also, this performance dip is important to consider when reporting performance given that kernels with GPU-initiated communication will cause increase in resource usage and thereby, decrease in occupancy. So for certain problem sizes, more warps of the MPI version of the kernels can run concurrently offsetting the performance benefits of GPU-initiated communication.

Finally, from matrix size of 4Kx4K onwards, as the interior compute time starts to dominate the overall running time, both versions converge in performance with the GPU Verbs doing marginally better by about 7% due to fused nature of its kernel enabling better reuse. This continues up to the largest input size of 32Kx32K that fits on the memory of Tesla P100 GPU. This shows that kernel rewriting needed to support communication from the 2D stencil application kernel does not add any performance overhead for large sizes, even though

GPU Verbs is designed to help only small sizes.

VI. RELATED WORK

There have been a fair number of contributions from vendors and researchers that are geared towards enabling efficient communication from GPUs. NVIDIA's GPUDirect technologies [8], [9] have helped reduce communication overheads by removing the need to copy data to CPU memory before it can be moved to other GPUs, within and across nodes. However, the CPU is still involved in initiating communication and synchronizing between computation and communication in the application. This demands a powerful CPU for best performance and incurs overheads that limit strong scaling. GPUDirect Async [10] addresses this by allowing GPU to trigger and synchronize network operations in order with compute kernels that are queued by the CPU. This will allow CPU to be put in low power states as computation and communication is progressed by the GPU. However, communication operations are scheduled only at kernel boundaries. Computation and communication phases have to be defined and queued from the CPU. This will result in multiple kernels and bulk synchronization phases whose overheads can dominate as applications are scaled strongly. In this paper, we consider communication from inside CUDA kernels as an alternative approach and show how it can be implemented using IB verbs developed for CUDA.

Several solutions have been proposed to enable efficient use of GPUs with programming models such as MPI and PGAS. Wang et. al. and Potluri et. al. [11], [12] has proposed CUDA-aware MPI with MVAPICH2 [13] which allows use of standard MPI interfaces for moving data from GPU or host memories. They take advantage of unified virtual addressing (UVA) feature from NVIDIA. Ashwin et. al. proposed use of datatype attributes to identify GPU memory in communication using standard MPI interfaces [14]. Potluri et. al. proposed CUDA-aware approach for OpenSHMEM [15]. The aforementioned works address CPU-initiated communication involving GPU device memory. Cunningham et.al have proposed the use of X10 to program GPUs and CPUs across clusters as part of their APGAS programming model [16]. Miyoshi et.al have contributed work that allows embedding MPI calls within GPU kernels in their FLAT GPU framework [17]. In our approach, we allow the GPU user to rely on the familiar CUDA programming model while using OpenSHMEM to express inter-GPU data movement inline with the CUDA model.

Researchers have implemented prototypes of IB verbs for GPUs in prior work. Oden et al. had observed that IB verbs are not suited for GPU-initiated communication [6]. Daoud et al. have shown ways to work around overheads and achieve better performance with IB verbs initiated on the GPU [7]. In this work, we push the state-of-the-art through designs that take into consideration the characteristics and features of the GPU. We discuss how these designs can be used in the OpenSHMEM communication model. We also discuss the port of a 2DStencil mini-app that is latency sensitive and is representative of a common category of HPC applications.

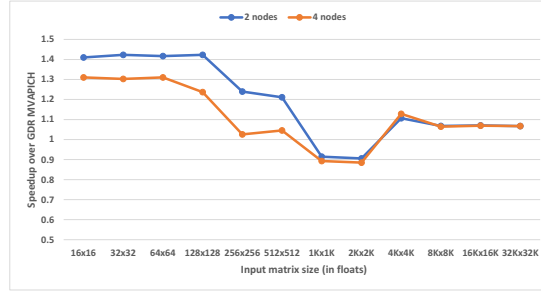


Fig. 9. Speedup of 2D stencil using GPU Verbs v/s using GDR MVAPICH

Gysi et al. [18] have presented dCUDA, a framework that provides GPU-side communication through CPU on-loading. In this work, we focus on GPU native implementation of IB verbs and their performance.

VII. CONCLUSION

In earlier work, we have shown how GPU-side implementation of OpenSHMEM can be used to achieve better application performance on GPUs connected through PCIe or NVLink. In this paper, we implemented IB verbs for Mellanox InfiniBand adapters in CUDA. We evaluated different design alternatives, taking into consideration the relaxed memory model, automatic memory access coalescing and thread hierarchy on the GPU addressing correctness issues that arise in these designs. We take advantage of these designs transparently or through API extensions in NVSHMEM. We presented a detailed evaluation using micro-benchmarks and a 2dstencil mini-app. On a single GPU using a single IB EDR adapter, we achieve a throughput of around 90 million messages per second. Speedups in the range of 23% to 42% are seen with a 2dstencil miniapp.

REFERENCES

- [1] "NVIDIA CUDA." [Online]. Available: <http://docs.nvidia.com/cuda>
- [2] Message Passing Interface Forum, <http://www.mpi-forum.org/>.
- [3] Open MPI, <https://www.open-mpi.org/>.
- [4] MVAPICH, <http://mvapich.cse.ohio-state.edu/>.
- [5] S. Tariq, "Lessons learned in improving scaling of applications on large gpu clusters," in *HPC Advisory Council Stanford Conference*, February 2003. [Online]. Available: http://www.hpcadvisorycouncil.com/events/2013/Stanford-Workshop/pdf/Presentations/Day%201/11_NVIDIA.pdf
- [6] *2014 IEEE International Parallel & Distributed Processing Symposium Workshops, Phoenix, AZ, USA, May 19-23, 2014*. IEEE Computer Society, 2014. [Online]. Available: <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=6967893>
- [7] F. Daoud, A. Watad, and M. Silberstein, "Gpurdma: Gpu-side library for high performance networking from gpu kernels," in *Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers*, ser. ROSS '16. New York, NY, USA: ACM, 2016, pp. 6:1–6:8. [Online]. Available: <http://doi.acm.org/10.1145/2931088.2931091>
- [8] NVIDIA, "Gpudirect," <https://developer.nvidia.com/gpudirect>, 2015.
- [9] —, "Gpudirect rdma," <http://docs.nvidia.com/cuda/gpudirect-rdma>, 2015.
- [10] D. Rossetti, "Gpudirect: Integrating the gpu with a network interface," in *GPU Technology Conference*, April 2015. [Online]. Available: <http://on-demand.gputechconf.com/gtc/2015/presentation/S5412-Davide-Rossetti.pdf>
- [11] H. Wang, S. Potluri, M. Luo, A. K. Singh, S. Sur, and D. K. Panda, "Mvapich2-gpu: Optimized gpu to gpu communication for infiniband clusters," *Comput. Sci.*, vol. 26, no. 3-4, pp. 257–266, Jun. 2011. [Online]. Available: <http://dx.doi.org/10.1007/s00450-011-0171-3>
- [12] S. Potluri, K. Hamidouche, A. Venkatesh, D. Bureddy, and D. K. Panda, "Efficient inter-node mpi communication using gpudirect rdma for infiniband clusters with nvidia gpus," in *Proceedings of the 2013 42Nd International Conference on Parallel Processing*, ser. ICPP '13. Washington, DC, USA: IEEE Computer Society, 2013, pp. 80–89. [Online]. Available: <http://dx.doi.org/10.1109/ICPP.2013.17>
- [13] MVAPICH, <http://mvapich.cse.ohio-state.edu/>.
- [14] A. M. Aji, J. Dinan, D. Buntinas, P. Balaji, W.-c. Feng, K. R. Bisset, and R. Thakur, "MPI-ACC: An Integrated and Extensible Approach to Data Movement in Accelerator-Based Systems," in *14th IEEE International Conference on High Performance Computing and Communications*, Liverpool, UK, June 2012.
- [15] S. Potluri, D. Bureddy, H. Wang, H. Subramoni, and D. K. Panda, "Extending openshmem for gpu computing," in *Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, ser. IPDPS '13. Washington, DC, USA: IEEE Computer Society, 2013, pp. 1001–1012. [Online]. Available: <http://dx.doi.org/10.1109/IPDPS.2013.104>
- [16] D. Cunningham, R. Bordawekar, and V. Saraswat, "Gpu programming in a high level language: Compiling x10 to cuda," in *Proceedings of the 2011 ACM SIGPLAN X10 Workshop*, ser. X10 '11. New York, NY, USA: ACM, 2011, pp. 8:1–8:10. [Online]. Available: <http://doi.acm.org/10.1145/2212736.2212744>
- [17] T. Miyoshi, H. Irie, K. Shima, H. Honda, M. Kondo, and T. Yoshinaga, "Flat: A gpu programming framework to provide embedded mpi," in *Proceedings of the 5th Annual Workshop on General Purpose Processing with Graphics Processing Units*, ser. GPGPU-5. New York, NY, USA: ACM, 2012, pp. 20–29. [Online]. Available: <http://doi.acm.org/10.1145/2159430.2159433>
- [18] T. Gysi, J. Baer, and T. Hoefler, "dCUDA: Hardware Supported Overlap of Computation and Communication," Nov. 2016, accepted at The International Conference for High Performance Computing, Networking, Storage and Analysis (SC'16).